CONCEPTOS Y PARADIGMAS DE LENGUAJES DE PROGRAMACIÓN

INTRODUCCIÓN Y EVALUACIÓN DE LENGUAJES

PROYECTO
“EL BARRIO VA A LA UNIVERSIDAD”
CONVOCATORIA COLABORADORES

- **19 años trabajando con la comunidad**
- **Capacitando a niños y jóvenes de diferentes barrios de la ciudad de La Plata y alrededores**
- **Acercando a los sectores más vulnerables a la Universidad, con el objetivo de que estos sectores puedan incorporar en sus imaginarios la posibilidad de continuar sus estudios en la Universidad.**
- **Formas de colaboración:**

  Colaborando en las capacitaciones.

  Colaborando con la producción del material

Introducción

Un lenguaje de programación es un lenguaje formal diseñado para expresar algoritmos y describir procesos computacionales que pueden ser ejecutados por una máquina.

Proporciona un conjunto de reglas sintácticas y semánticas que permiten especificar instrucciones que serán interpretadas o compiladas en operaciones ejecutables por un sistema informático.

Introducción

Un lenguaje de programación puede seguir o respetar uno o varios paradigmas de programación, que guían y dictan cómo se escriben y conceptualizan los programas.

Un paradigma de programación indica una forma de pensar la programación. Define un modelo, métodos de realizar cómputos y la manera en que se deben estructurar y organizar las tareas que debe llevar a cabo un programa.

Ejemplos:

Se tiene el **paradigma imperativo** como la programación estructurada, la orientada a objetos; el **paradigma declarativo** que incluye la programación funcional, la programación lógica.

Principales componentes

- Sintaxis
- Semántica
- Sistema de tipos
- Modelo de ejecución
- Mecanismos de abstracción

Principales componentes

- **Sintaxis**

  Es el **conjunto de reglas sintácticas y gramaticales** que determinan cómo se deben estructurar las instrucciones del lenguaje.

  Define la **forma válida de los programas**.

  Suele describirse mediante **gramáticas formales** (por ejemplo, BNF o EBNF).

Principales componentes

- **Sintaxis**
- **Semántica**

  Define el **significado** de las construcciones sintácticas.

  Semántica operacional: describe cómo se ejecuta un programa paso a paso.

  Semántica denotacional: asocia construcciones del lenguaje con objetos matemáticos.

  Semántica axiomática: permite razonar formalmente sobre la corrección de programas.

Principales componentes

- **Sintaxis**
- **Semántica**
- **Sistema de tipos**

Establece **reglas** sobre los tipos de datos y su uso.

Funciones principales:

detectar errores

garantizar consistencia

mejorar seguridad del programa

Ejemplos:

tipado estático vs. dinámico

tipado fuerte vs. débil

Principales componentes

- **Sintaxis**
- **Semántica**
- **Sistema de tipos**
- **Modelo de ejecución**

  Define **cómo se ejecuta un programa**.

  Compilado: traducido a código máquina antes de ejecutarse

  Interpretado: ejecutado por un intérprete y traducido en tiempo de ejecución

  Híbrido → bytecode + máquina virtual

Principales componentes

- **Sintaxis**
- **Semántica**
- **Sistema de tipos**
- **Modelo de ejecución**
- **Mecanismos de abstracción**

  Proporcionan **abstracciones computacionales y de complejidad**

  Ejemplos:

  Estructuras de control
  Estructuras de datos
  Módulos y funciones
  Clases y objetos
  Concurrencia

Objetivos

- Adquirir habilidad de **comprender** y **evaluar** lenguajes, identificando los conceptos más importantes de cada uno de ellos, sus límites y posibilidades.
- Habilidad para **elegir**, para **diseñar**, **implementar** o **utilizar** un lenguaje de programación.
- Enfatizar la **abstracción** como la mejor forma de manejar la complejidad de objetos y fenómenos

"El valor de un lenguaje se debe juzgar según la forma en que afecta la producción de Software y a la facilidad con la que puede integrarse a otras herramientas"

Estudio de Conceptos de Lenguajes de Programación

- **Aumentar la capacidad para producir software.**
- **Mejorar el uso del lenguaje**
- **Elegir mejor un lenguaje**
- **Facilitar el aprendizaje de nuevos lenguajes**
- **Facilitar el diseño e implementación de lenguajes**

Criterios de evaluación

Criterios para analizar la calidad del diseño de un lenguaje de programación:

- **Simplicidad y legibilidad**
- **Claridad en los bindings**
- **Confiabilidad**
- **Soporte**
- **Abstracción**
- **Ortogonalidad**
- **Eficiencia**

Criterios de evaluación

Textos obtenidos de trabajos finales de promoción

Pregunta a docentes de Cátedra de Lenguaje C:
Suele observarse que el código de C **no es legible** ¿Cree que esto se debe a la naturaleza del lenguaje o a malos hábitos de programación? ¿Cuál cree que sea la causa?

Rta 1:
*La sintaxis de C permite que pueda escribirse **código muy compacto**. Los alumnos deberían salir preparados para **leer código de otros sin problemas**. Muchos de los proyectos más grandes escritos en C tienen estrictas normas de estilo y esto permite que a pesar de ser compacto, el código **sea suficientemente expresivo** para todos.*

Rta 2:
A la naturaleza del lenguaje sobre todo. El permitir código conciso y…..

Pregunta a docentes de Cátedra lenguaje Ruby:
¿Qué opina de la sintaxis de Ruby?

Rta:
*La sintaxis de Ruby es, a mi parecer, **simple y elegante.** ..... y por otro lado es muy fácil desarrollar con él porque es muy conciso **y preciso** a la hora de implementar cualquier clase de funcionalidad*

Criterios de evaluación

Pregunta a docentes de Cátedra lenguaje Ruby:
¿Usted considera que Ruby es un buen ejemplo de un lenguaje que presenta **Ortogonalidad?.** ¿A qué se debe?

Rta:
*A mi parecer sí. El simple hecho de que **toda sentencia del lenguaje sea una expresión** (incluso las estructuras de control) permite realizar combinaciones y/o composiciones sin ninguna clase de limitación. Otro factor que favorece la ortogonalidad es que casi todo en este lenguaje son objetos; no recuerdo fácilmente casos en los cuales haya tenido problemas por encontrarme con algo que no sea/se comporte como un objeto.*

Pregunta a docentes de Cátedra lenguaje Python:
¿Piensa que el lenguaje Python es indicado para iniciar a los alumnos en la programación?

Rta:
*Si, Python es **un lenguaje simple y fácil de enseñar**. Es un lenguaje de tipado dinámico pero es fuertemente tipado, Es un lenguaje **ortogonal** porque se pueden combinar sus componentes. Su código es **legible**, **confiable** por ser fuertemente tipado y proveer manejo de excepcioenes,*

En las respuestas se pueden observar criterios de evaluación aplicados, relacionados con: **Simplicidad, Ortogonalidad, Confiabilidad, Expresividad,** etc.

Simplicidad y Legibilidad

Los lenguajes de programación simples y legibles:

- producen programas fáciles de escribir y de leer.
- fáciles a la hora de aprenderlo o enseñarlo
- con sintaxis clara y pocas reglas complejas
- evitan ambigüedades

Permiten:

- **en escritura (writability):** expresividad, rapidez al programar
- **en legibilidad (readability):** favorece el mantenimiento del software, trabajo en equipo y revisión de código

Ejemplo de cuestiones que atentan contra esto:

- muchas componentes elementales, operadores sobrecargados
- el mismo concepto semántico – distinta sintaxis
- distintos conceptos semánticos - la misma notación sintáctica

Claridad en los bindings

Es la **asociación entre un nombre y una entidad del programa**.

Ejemplos de bindings:

- nombre de variable y su valor
- nombre de función y su implementación

La claridad en los bindings implica:

- saber **cuándo** y **hasta cuándo** ocurre la asociación
- **reglas de scope** (alcance) claras y no ambiguas

Momentos posibles:

- definición del lenguaje
- implementación del lenguaje
- en escritura del programa
- compilación
- cargado del programa
- en ejecución

Tipos de binding:

estático (en compilación)

dinámico (en ejecución)

Confiabilidad

La confiabilidad está relacionada con la **seguridad.**

Un lenguaje es confiable cuando:

- evita errores comunes
- ofrece **tipado fuerte**
- **detecta errores en compilación**
- **previene** comportamientos inesperados

Implica

- **Chequeo de tipos:** Cuanto antes se encuentren errores menos costoso resulta realizar los arreglos que se requieran.
- **Manejo de excepciones:** La habilidad para interceptar errores en tiempo de ejecución, tomar medidas correctivas y continuar.

Soporte

Un lenguaje con buen soporte facilita:

- el aprendizaje
- el desarrollo
- el mantenimiento

El soporte puede consistir en:

- compiladores e intérpretes
- bibliotecas
- frameworks
- herramientas de depuración
- comunidad de desarrolladores
- documentación
- tutoriales

Abstracción

Permite en programación:

- representar conceptos complejos mediante estructuras más simples
- definir y usar estructuras u operaciones complicadas de manera que sea posible ignorar muchos de los detalles.
- ocultar detalles de implementación
- reutilizar código.
- manejar complejidad

Ejemplos:

- **en abstracción de datos:** tipo de datos abstractos, clases
- **abstracción en procesos:** funciones, módulos, interfaces

Ortogonalidad

La **ortogonalidad** significa que las características del lenguaje funcionan de manera **independiente** y **consistente** entre sí.

Significa en programación:

- que un conjunto pequeño de **constructores primitivos** puedan combinarse **sin restricciones** mediante un conjunto consistente y uniforme de **reglas de combinación**.
- cada combinación es legal y con sentido.

Ejemplos de problemas de ortogonalidad:

- "funciona sólo con enteros”
- "en este caso no se puede usar dentro de una función”
- “de este tipo no se puede retornar“

La ortogonalidad reduce:

- excepciones en las reglas
- complejidad del lenguaje

Ortogonalidad

https://www.mheducation.es/bcv/guide/capitulo/8448148681.pdf

3.4 Devolución de valores

Una función en C sólo puede devolver un valor. Para devolver dicho valor, se utiliza la palabra reservada *return* cuya sintaxis es la siguiente:

```
return <expresión>;
```

Donde *<expresión>* puede ser cualquier tipo de dato salvo un array o una función. Además, el valor de la expresión debe coincidir con el tipo de dato declarado en el prototipo de la función. Por otro lado, existe la posibilidad de devolver múltiples valores mediante la utilización de punteros o estructuras. Dentro de una función pueden existir varios *return* dado que el programa devolverá el control a la sentencia que ha llamado a la función en cuanto encuentre la primera sentencia *return*. Si no existen *return*, la ejecución de la función continúa hasta la llave del final del cuerpo de la función *(}}*. Hay que tener en cuenta que existen funciones que no devuelven ningún valor. El tipo de dato devuelto por estas funciones puede ser *void*, considerado como un tipo especial de dato. En estos casos, la sentencia *return* se puede escribir como *return* o se puede omitir directamente. Por ejemplo:

```
void imprime_cabecera();
```

Eficiencia

La **eficiencia** hace referencia al **uso eficiente de los recursos** computacionales por parte del programa generado por el lenguaje de programación.

Afecta cuestiones como:

- tiempo y espacio
- esfuerzo humano
- optimización

Puede medirse:

- tiempo de ejecución
- uso de memoria
- consumo de CPU

Sintaxis

Sintaxis y semántica

Un lenguaje de programación ofrece una **notación formal** para describir algoritmos a ser ejecutados en una computadora. Requiere de una sintaxis –si es válido el programa- y semántica –qué significa- tanto para la **programación** como para su **traducción**.

Sintaxis: conjunto de reglas que definen cómo componer letras, dígitos y otros caracteres para formar los programas.

Semántica: conjunto de reglas **para dar significado** a los programas sintácticamente válidos.

```
 v: array [1..10] of integer;     -------- en Pascal
y
 int v[10]; ---------------------- en C
```

Sintaxis

Permite escribir programas **correctos y válidos** sintácticamente. Establece reglas para combinar componentes básicas, llamadas **word** y formar sentencias y programas.

Elementos que la componen:

- alfabeto o conjunto de caracteres
- identificadores
- operadores
- palabra clave y palabra reservada
- comentarios y uso de blancos

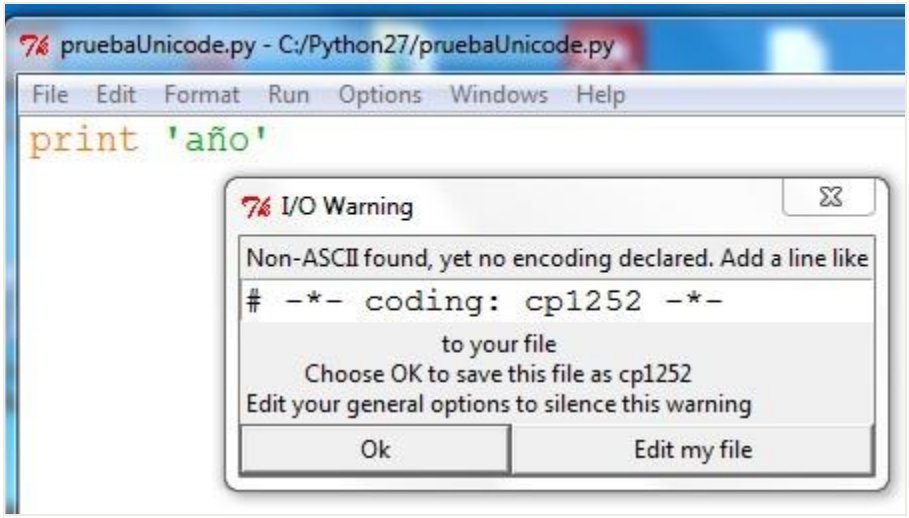
Contempla cuestiones:

- legibilidad
- verificabilidad
- traducción
- falta de ambigüedad

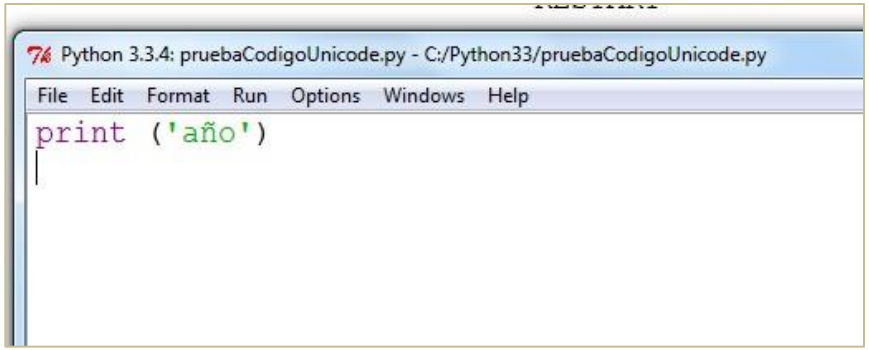
Sintaxis

Alfabeto

- con qué conjunto de caracteres se trabaja
- la secuencia de bits que compone cada carácter determina la implementación.



Python 2.x



Python 3.x

Sintaxis

Alfabeto

Latin-1 (ISO-8859-1: Western European)

1	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0x0	nul	stx	sot	etx	eot	enq	ack	bel	bs	ht	lf	vt	ff	cr	so	si
0x1	dle	dc1	dc2	dc3	dc4	nak	syn	etb	can	em	sub	esc	fs	gs	rs	us
0x2	sp	!	"	#	$	%	&	'	(	)	*	+	,	-	.	/
0x3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
0x4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
0x5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
0x6	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
0x7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	del
0x8	pad	hop	bph	nbh	ind	nel	ssa	esa	hts	htj	vts	pld	plu	ri	ss2	ss3
0x9	dcs	pu1	pu2	sts	cch	mw	spa	epa	sos	sgci	sci	csi	st	osc	pm	apc
0xa	nbsp	¡	¢	£	¤	¥	¦	§	¨	©	ª	«	¬	shy	®	¯
0xb	°	±	²	³	´	µ	¶	·	¸	¹	º	»	¼	½	¾	¿
0xc	À	Á	Â	Ã	Ä	Å	Æ	Ç	È	É	Ê	Ë	Ì	Í	Î	Ï
0xd	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß
0xe	à	á	â	ã	ä	å	æ	ç	è	é	ê	ë	ì	í	î	ï
0xf	ð	ñ	ò	ó	ô	õ	ö	÷	ø	ù	ú	û	ü	ý	þ	ÿ

El código ASCII

sigla en inglés de American Standard Code for Information Interchange
(Código Estadounidense Estándar para el Intercambio de Información)

Caracteres de control ASCII

DEC	HEX	Símbolo ASCII	
00	00h	NULL	(carácter nulo)
01	01h	SOH	(inicio encabezado)
02	02h	STX	(inicio texto)
03	03h	ETX	(fin de texto)
04	04h	EOT	(fin transmisión)
05	05h	ENQ	(enquiry)
06	06h	ACK	(acknowledgement)
07	07h	BEL	(timbre)
08	08h	BS	(retroceso)
09	09h	HT	(tab horizontal)
10	0Ah	LF	(salto de línea)
11	0Bh	VT	(tab vertical)
12	0Ch	FF	(form feed)
13	0Dh	CR	(retorno de carro)
14	0Eh	SO	(shift Out)
15	0Fh	SI	(shift In)
16	10h	DLE	(data link escape)
17	11h	DC1	(device control 1)
18	12h	DC2	(device control 2)
19	13h	DC3	(device control 3)
20	14h	DC4	(device control 4)
21	15h	NAK	(negative acknowle.)
22	16h	SYN	(synchronous idle)
23	17h	ETB	(end of trans. block)
24	18h	CAN	(cancel)
25	19h	EM	(end of medium)
26	1Ah	SUB	(substitute)
27	1Bh	ESC	(escape)
28	1Ch	FS	(file separator)
29	1Dh	GS	(group separator)
30	1Eh	RS	(record separator)
31	1Fh	US	(unit separator)
127	7Fh	DEL	(delete)

Caracteres ASCII imprimibles

DEC	HEX	Símbolo	DEC	HEX	Símbolo	DEC	HEX	Símbolo
32	20h	espacio	64	40h	@	96	60h	`
33	21h	!	65	41h	A	97	61h	a
34	22h	"	66	42h	B	98	62h	b
35	23h	#	67	43h	C	99	63h	c
36	24h	$	68	44h	D	100	64h	d
37	25h	%	69	45h	E	101	65h	e
38	26h	&	70	46h	F	102	66h	f
39	27h	'	71	47h	G	103	67h	g
40	28h	(	72	48h	H	104	68h	h
41	29h	)	73	49h	I	105	69h	i
42	2Ah	*	74	4Ah	J	106	6Ah	j
43	2Bh	+	75	4Bh	K	107	6Bh	k
44	2Ch	,	76	4Ch	L	108	6Ch	l
45	2Dh	-	77	4Dh	M	109	6Dh	m
46	2Eh	.	78	4Eh	N	110	6Eh	n
47	2Fh	/	79	4Fh	O	111	6Fh	o
48	30h	0	80	50h	P	112	70h	p
49	31h	1	81	51h	Q	113	71h	q
50	32h	2	82	52h	R	114	72h	r
51	33h	3	83	53h	S	115	73h	s
52	34h	4	84	54h	T	116	74h	t
53	35h	5	85	55h	U	117	75h	u
54	36h	6	86	56h	V	118	76h	v
55	37h	7	87	57h	W	119	77h	w
56	38h	8	88	58h	X	120	78h	x
57	39h	9	89	59h	Y	121	79h	y
58	3Ah	:	90	5Ah	Z	122	7Ah	z
59	3Bh	;	91	5Bh	[	123	7Bh	{
60	3Ch	<	92	5Ch	\	124	7Ch	|
61	3Dh	=	93	5Dh	]	125	7Dh	}
62	3Eh	>	94	5Eh	^	126	7Eh	~
63	3Fh	?	95	5Fh	_			

elCodigoASCII.com.ar

Sintaxis

Alfabeto

Cyrillic (ISO-885

C	0	1	2	3	4	5	6
0x0	nul	stx	sot	etx	eot	enq	ack
0x1	dle	dc1	dc2	dc3	dc4	nak	syn
0x2	sp	!	"	#	$	%	&
0x3	0	1	2	3	4	5	6
0x4	@	A	B	C	D	E	F
0x5	P	Q	R	S	T	U	V
0x6	`	a	b	c	d	e	f
0x7	p	q	r	s	t	u	v
0x8	pad	hop	bph	nbh	ind	nel	ssa
0x9	dcs	pu1	pu2	sts	cch	mw	spa
0xa	nbsp	Ё	Ђ	Ѓ	Є	S	I
0xb	А	Б	В	Г	Д	Е	Ж
0xc	Р	С	Т	У	Ф	Х	Ц
0xd	а	б	в	г	д	е	ж
0xe	р	с	т	у	ф	х	ц
0xf	№	ё	ђ	ѓ	є	s	і

Arabic (ISO-8859-6)

A	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
0x0	nul	stx	sot	etx	eot	enq										
0x1	dle	dc1	dc2	dc3	dc4	nak										
0x2	sp	!	"	#	$	%										
0x3	٠	١	٢	٣	٤	٥										
0x4	@	A	B	C	D	E										
0x5	P	Q	R	S	T	U										
0x6	`	a	b	c	d	e										
0x7	p	q	r	s	t	u										
0x8	pad	hop	bph	nbh	ind	nel										
0x9	dcs	pu1	pu2	sts	cch	mw										
0xa	nbsp	XX	XX	XX	¤	XX										
0xb	XX	XX	XX	XX	XX	XX										
0xc	XX	،	أ	آ	ؤ	إ										
0xd	ذ	ر	ز	س	ش	ص										
0xe	ـ	ف	ق	ك	ل	م										
0xf	ً	ٌ	ٍ	XX	XX	XX										

Greek (ISO-8859-7)

G	0	1	2	3	4	5	6	7	8	9	a
0x0	nul	stx	sot	etx	eot	enq	ack	bel	bs	ht	lf
0x1	dle	dc1	dc2	dc3	dc4	nak	syn	etb	can	em	sub
0x2	sp	!	"	#	$	%	&	'	(	)	*
0x3	0	1	2	3	4	5	6	7	8	9	:
0x4	@	A	B	C	D	E	F	G	H	I	J
0x5	P	Q	R	S	T	U	V	W	X	Y	Z
0x6	`	a	b	c	d	e	f	g	h	i	j
0x7	p	q	r	s	t	u	v	w	x	y	z
0x8	pad	hop	bph	nbh	ind	nel	ssa	esa	hts	htj	vts
0x9	dcs	pu1	pu2	sts	cch	mw	spa	epa	sos	sgci	sci
0xa	nbsp	‘	’	£	XX	XX	¦	§	¨	©	XX
0xb	°	±	²	³	΄	΄΄	Ά	·	Έ	Η	Ι
0xc	ΐ	Α	Β	Γ	Δ	Ε	Ζ	Η	Θ	Ι	Κ
0xd	Π	Ρ	XX	Σ	Τ	Υ	Φ	Χ	Ψ	Ω	Ϊ
0xe	ΰ	α	β	γ	δ	ε	ζ	η	θ	ι	κ
0xf	π	ρ	ς	σ	τ	υ	φ	χ	ψ	ω	ϊ

UNICODE

Sintaxis

Identificadores

- Por ejemplo conformado por letras y dígitos, que deben comenzar con una letra.
- Si se restringe la longitud se pierde legibilidad.

Operadores

- Uniformes: con los operadores de suma, resta, etc. la mayoría de los lenguajes utilizan +, -.
- Poco uniformes: otros operadores no hay tanta uniformidad (**|^).

Comentarios

- para hacer los programas más legibles.

“El código es leído muchas más veces de lo que es escrito”.

Guido Van Roussen (creador de Python).

Sintaxis

Palabra clave y palabra reservada

Array do else if

- **Palabra clave o keywords:** son palabras claves que tienen un significado dentro de un contexto.
- **Palabra reservada:** son palabras claves que además no pueden ser usadas por el programador como identificador de otra entidad.

Ventajas de su uso: Hacen los programas más legibles y permiten una rápida traducción

Ejemplos de lenguajes con uso de palabras reservadas:

- **C ej.:**auto, break, case, char, const, continue, default, do ,double, else, enum, extern, float, for, goto, if, int, etc
- **Pascal ej.:** absolute, and, array, begin, const, div, do, downto, else, if, in, label, mod, not,of, packed, procedure, record, set, shr, then,to, unit, uses, var, while, xor, etc

Sintaxis

Python: las palabras reservadas y sus versiones…

Python 2.7.9 Shell

File Edit Shell Debug Options Windows Help

```
Python 2.7.9 (default, Dec 10 2014, 12:24:55) [MSC v.1500 32 bit (Intel)]
on win32
Type "copyright", "credits" or "license()" for more information.
>>> True='hola'
>>> print True
hola
>>>
```

Ln: 16 Col: 4

Python 3.6.4 Shell

File Edit Shell Debug Options Window Help

```
Python 3.6.4 (v3.6.4:d48eceb, Dec 19 2017, 06:04:45) [MSC v.1900 32 bit (Intel)]
 on win32
Type "copyright", "credits" or "license()" for more information.
>>> print=9
>>> True='hola'
SyntaxError: can't assign to keyword
>>>
```

Ln: 131 Col: 4

Sintaxis

Python: las palabras reservadas y sus versiones…

En las versiones 2.x el lenguaje cuenta con 31 palabras reservadas:

```
and as assert break class continue def del elif else except
exec finally for from global if import in is lambda not or
pass print raise return try while with yield
```

En la versión 3.x se quitaron de la lista **exec y print** han sido removidas, ya que ahora se presentan como funciones incorporadas por defecto.
Se han aadido: los térmiinos nonlocal, True, False y None

Por lo tanto, la lista de *keywords* en Python 3 resulta ser la siguiente.

```
False None True and as assert break class continue def del
elif else except finally for from global if import in is
lambda nonlocal not or pass raise return try while with yield
```

Sintaxis

Estructura sintáctica

Vocabulario o words

- Conjunto de caracteres y palabras necesarias para construir expresiones, sentencias y programas. Ej: identificadores, operadores, palabras claves, etc.
- Las words no son elementales se construyen a partir del alfabeto.

Expresiones

- Son funciones que a partir de un conjunto de datos devuelven un resultado.
- Son bloques sintácticos básicos a partir de los cuales se construyen las sentencias y programas

Sentencias

- Componente sintáctico más importante.
- Tiene un fuerte impacto en la facilidad de escritura y legibilidad

Sintaxis

Reglas léxicas y sintácticas

Reglas léxicas

- conjunto de reglas para formar las **“word”**, a partir de los caracteres del alfabeto.
- por ej. diferencias entre mayúsculas y minúsculas, símbolo de distinto != o <>

Reglas sintácticas

- conjunto de reglas que definen cómo son las **sentencias** y **expresiones**.
- por ej. En C el if no lleva then, pero en Pascal sí.

Sintaxis

Tipo de sintaxis

Abstracta

- Se refiere básicamente a la estructura.

Concreta

- Se refiere a la parte léxica.

Pragmática

- Se refiere al uso práctico.

Sintaxis

Ejemplo de sintaxis concreta y abstracta

while (x!= y)	*while x<>y do*
{	*begin*
----------	*--------*
};	*end*
(En C)	**(En Pascal)**

Uso de paréntesis

Forma de encerrar un bloque

Símbolo de distinto

- Son diferentes respecto a la **sintaxis concreta**, porque existen diferencias léxicas entre ellas
- Son iguales respecto a la **sintaxis abstracta**, ya que ambas tienen la misma estructura

while condición bloque

Sintaxis

Ejemplo de sintaxis pragmática

Ej1.

<> es más legible que !=

Ej2.

En C y Pascal {} o begin-end pueden omitirse si el bloque está compuesto por una sola sentencia

while (x!=y) x=y+1

Pragmáticamente puede conducir a error ya que si se necesitara agregar una sentencia debe agregarse el begin end o las {}

(es decir que no define una única forma de escribirse)

Sintaxis

Definición de la sintaxis

Se necesita **una descripción finita** para definir un conjunto infinito (conjunto de todos los programas bien escritos)

Formas para definir la sintaxis:

- Lenguaje natural. Ej.: Fortran
- **Gramáticas libres de contexto:**

  Backus y Naun Form: BNF Ej: Algol

  EBNF

  Diagramas sintácticos equivalentes a BNF pero más intuitivos

Sintaxis

BNF (Backus Naun Form)

- Es una notación formal para describir la sintaxis
- Es un metalenguaje
- Utiliza metasímbolos
  - **< > ::= |**
- Define las reglas por medio de "producciones"

Ejemplo:

< digito **>** **::=** 0|1|2|3|4|5|6|7|8|9

No terminal

Metasímblo

Se define como **Terminales**

Sintaxis

Gramática

- Conjunto de reglas finita que define un conjunto infinito de posibles sentencias válidas en el lenguaje.
- Una gramática está formada por una 4-tupla

$$G = (N, T, S, P)$$

Conjunto de símbolos no terminales

Conjunto de símbolos terminales

Símbolo distinguido de la gramática que pertenece a N

Conjunto de producciones

Sintaxis

Parsing y Árboles sintácticos

- Es una oración sintácticamente incorrecta
- No todas las oraciones que se pueden armar con los terminales son válidas
- Se necesita de un **Método de análisis (reconocimiento)** que permita determinar si un string dado es válido o no en el lenguaje: **Parsing.**

El **parse,** para cada sentencia construye un **"árbol sintáctico o árbol de derivación"**

Sintaxis

Árboles sintácticos

Dos maneras de construirlo:

- **Método bottom-up**
  - De izquierda a derecha
  - De derecha a izquierda
- **Método top-dow**
  - De izquierda a derecha
  - De derecha a izquierda

Sintaxis

Árbol de derivación

Ejemplo top-down de izquierda a derecha

N={<oración> => <sujeto><predicado>
<predicado> => <verbo> <art> <objeto>
<sujeto> => Juan | María
<art> => un | una
<objeto> => mascota | perro | gato}

=> Juan <predicado>
=> Juan <verbo><objeto>
=> Juan adopta <objeto>
=> Juan adopta <art><sustan>
=> Juan adopta un <sustan>
=>Juan adopta un perro

- Los compiladores utilizan el **parse canónico** que es el **bottom-up** de **izquierda a derecha.**

Sintaxis

Árbol de derivación

Otro ejemplo

- Expresiones simples de uno y dos términos
- Posibles operaciones: + / * y –
- Solo los operandos A, B y C
- Ejemplo de expresiones válidas:

•A

•A+B

•A-C

etc.

Sintaxis

Producciones recursivas

- Son las que hacen que el conjunto de sentencias descrito sea infinito
- **Ejemplo de producciones recursivas:**

<natural> ::= <digito> | <digito><digito> | ..... | <digito>.....<digito>

- Si lo planteamos recursivamente

GN = (N, T, S, P)

N = { <natural>, <digito> } T = { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 }

S = <natural>

P = {<natural> ::= <digito> | <digito> <natural>,

<digito> ::= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9}

Cualquier gramática que tiene una producción recursiva describe un **lenguaje infinito**.

Sintaxis

Producciones recursivas

- **Regla recursiva por la izquierda**
  - La asociatividad es por la izquierda
  - El símbolo no terminal de la parte izquierda de una regla de producción aparece al comienzo de la parte derecha
- **Regla recursiva por la derecha**
  - La asociatividad es por la derecha
  - El símbolo no terminal de la parte izquierda de una regla de producción aparece al final de la parte derecha

Sintaxis

Gramáticas ambiguas

Una gramática es **ambigua** si una sentencia puede derivarse de más de una forma

G= (N, T, S, P)

N = { <id>, <exp>, <asig>} T = {

A,B,C, +,*,-,/,:=}

S = <asig>

P1 = {

<asig> ::= <id> := <exp>

<exp> ::= <exp>+<exp>|<exp>*<exp>|<exp>- <exp>|

<exp>/ <exp>|<id>

recursión

<id> ::= A | B | C

}

Sintaxis

Subgramáticas

- Sea la gramática para identificadores GI = (N, T, S, P)

N = { <id>, <letra>, <digito>, <otro>}

T = { A, ..., Z, 0, ..., 1}

S = <id>

P = { <id> ::= <letra> | <letra><otro>,

<otro>::= <letra> | <digito> | <letra><otro>

| <digito><otro>,

<letra> :: A | B | C |.....| Z

<digito> ::= 0 | 1 | 2 | | 9 }

- Para definir la gramática GE de expresiones, se puede utilizar la gramática de números y de identificadores.
- GE se definiría utilizando las **sub gramáticas** GN y GI

La filosofía de composición es la forma en que trabajan los compiladores

Sintaxis

Gramáticas libres de contexto y sensibles al contexto

int y:

y= 22.75

- La gramática **libre de contexto** no puede verificar:
    - si x fue **declarada antes**
    - si los **tipos son compatibles**
- No considera lo que se define antes ni después.
- Según nuestra gramática son sentencias sintácticamente válidas, aunque puede suceder que a veces no lo sea semánticamente.

Sintaxis

Gramáticas libres de contexto y sensibles al contexto

Gramática libre de contexto (CFG)

- es aquella en la que no realiza un análisis de contexto.
- describen las sintaxis de los lenguajes como C, Java, Python

Gramática sensible al contexto (CSG)

- se realizan verificaciones que consideran el **contexto del programa** (Algol 68)
- se manejan en la fase del **análisis semántico** del compilador, para detectar ciertas restricciones semánticas.
- se pueden utilizar en IA en aplicaciones de machine learning, reconocimiento de voz, en lingüística para especificar reglas para lenguaje natural

Sintaxis

EBNF BNF extendido

- EBNF es una **gramática libre de contexto**
- Simplifica la escritura
- Elimina la recursión y la reemplaza por **reiteración**

Meta símbolos incorporados:

[ ] elemento optativo puede o no estar

(|) selección de una alternativa

{}repetición

*** 0 o más veces + 1 o más veces**

Sintaxis

Ejemplo con EBNF

Definición números enteros en BNF y en EBNF

BNF

<enterosig> ::= + <entero> | - <entero> | <entero>

<entero> ::= <digito>| <entero><digito>

Recursión

EBNF

<enterosig>::= **[**(+|-)**]** <digito>{<digito>}*

Eliminó la recursión y es más fácil de entender

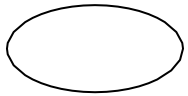
Sintaxis

Diagramas sintácticos (CONWAY):

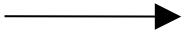
- Es un grafo sintáctico o carta sintáctica
- Cada diagrama tiene una entrada y una salida, y el camino determina el análisis.
- Cada diagrama representa una regla o producción
- Para que una sentencia sea válida, debe haber un camino desde la entrada hasta la salida que la describa.
- Se visualiza y entiende mejor que BNF o EBNF

Sintaxis

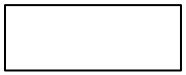
Diagramas sintácticos (CONWAY)



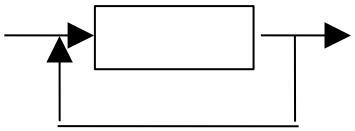
Terminales



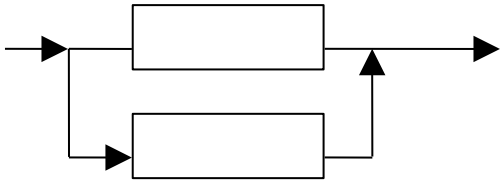
Flujo



No terminales



Repetición



Selección

Ej:

Sintaxis

Pensar:

Cómo definir una gramática para una expresión con operandos del tipo identificador y números, que refleje el orden de prioridades de las operaciones.